PULSERUST SERVER

Plugin Performance Audit

PulseDailyGems · Kits

Prepared: 19 June 2026 · Analyst: Claude — High-Performance Rust Plugin Engineer

EXECUTIVE SUMMARY

Both plugins were authored by the same developer and share the same critical architectural defect: **synchronous main-thread disk I/O executed on every player action**. On a high-population server running major raid events, this manifests as recurring main-thread stalls, garbage-collection spikes, and burst construction-packet queues visible in Carbon profiler. The Kits plugin is the primary offender: it calls `_storage.Save()` on **every single kit redemption** — a full binary serialise + 5-deep backup-file cascade on the main thread. PulseDailyGems does the same on every gem claim. When six sibling plugins with the same pattern all fire during `OnServerSave`, the stalls compound. The UI rendering layers of both plugins are well-engineered and are **not** the problem.

Finding	Plugin(s)	Severity	Main-Thread Impact
Save-on-every-redeem / claim	Kits + DailyGems	CRITICAL	HIGH — blocks 10–80 ms per event
OnServerSave stacking (×6 plugins)	All siblings	CRITICAL	HIGH — cascades into save-frame freeze
GetLastClaim write-on-read / unbounded dict growth	DailyGems	HIGH	Grows save size every panel open
GetOrCreateKitData creates entries on panel build	Kits	HIGH	Unbounded _usersData growth → heavier serialise
No dirty flag — saves identical state repeatedly	Kits + DailyGems	HIGH	Wastes all disk and CPU on unchanged data
String alloc in save path (hot with above bugs)	Both	MEDIUM	GC pressure compound during claim storms
EncodeToPNG round-trip on image load	Both	LOW	One-time CPU spike on plugin load
1.2 MB uncompressed banner image	DailyGems	LOW	Client connect-storm bandwidth only
PlayerHasGroupPermission calls permission.UserHasPermission in overlay loop	Kits	MEDIUM	Per-open overhead; masked by sig cache

1. Kits — Critical Findings

Plugin: Kits v2.0.2 · Author: Mevent

CRITICAL	[K-01]	**Synchronous disk save on every kit redemption**	*Kits*	Line 5209
Problem	GiveKitToPlayer() calls `_storage.Save(_storagePath)` synchronously on the main thread after every single kit redeem. Save() performs: (1) full protobuf serialisation of **all** player data into a pooled BufferStream, (2) ShiftBackups() — a 5-deep cascade of File.Exists / File.Move / File.Copy / File.Delete, (3) File.OpenWrite + fs.Write for the temp file, (4) File.Copy(tmp, path, true), (5) File.Delete(tmp). That is **10–15 blocking syscalls per redemption**. During a wipe-day kit storm with 80+ concurrent players each opening kits, these stalls queue serially on the main thread. Each call also serialises the entire `_usersData` dictionary, whose size grows unboundedly (see K-02), making each save progressively heavier.		
Fix	Introduce a dirty flag. Mark dirty in UpdatePlayerData(); do **not** touch disk. Flush on OnServerSave, Unload, and a timer.Every(120f) safety interval. Serialise to byte[] on the main thread (fast — no I/O), then hand to ThreadPool for all file ops. Unload must flush synchronously since the plugin is shutting down. This collapses a 80-player kit storm from 80×15 blocking syscalls to a handful of background writes.		
Note	*buyclass (line 2037) also calls _storage.Save() synchronously — same fix applies there.*		

CRITICAL	[K-02]	**GetOrCreateKitData() silently creates player records on every UI panel build**	*Kits*	Lines 5239, 5266, 5362, 5893
Problem	`GetOrCreateKitData()` and `GetOrCreate()` are called from: ComputeKitCd, EmitUsesLabel, AppendUsesText, AppendUsesColor, CanPlayerRedeemKit, ProcessAutoKit, CanSeeKit. Many of these execute every time a player opens or switches tabs in the kit panel. If the player has no record in `_usersData`, the method creates one and inserts it permanently. Over a wipe on a 100-player server, every player who opens the kits UI gets a `PlayerData` entry inserted, and for every kit, a nested `KitData` entry. This grows `_usersData` unboundedly — which means the protobuf serialisation in Save() gets progressively slower throughout the wipe, and the save file grows from kilobytes to megabytes of empty/zero records.		
Fix	Split read paths from write paths. Create two variants: `GetOrLoadKitData()` (already exists — use it for UI/display paths, returns null for unknown players) and `GetOrCreateKitData()` (only for actual redemption/mutation paths). Audit every callsite: CanSeeKit (5893), ComputeKitCd (2889), EmitUsesLabel (344), AppendUsesText (2397), AppendUsesColor (2430), GetAutoKits (5914) — all must use the load-only variant. Only GiveKitToPlayer / UpdatePlayerData / ProcessAutoKit / SetPlayerKitUses should create entries.		
Note	*Without this fix, a 4-hour play session on a 100-player server can produce 100 × (kit_count) zero-value KitData records. With 20 kits that is 2,000 empty records being serialised and backed up on every save.*		

HIGH	[K-03]	No dirty flag — identical state written to disk repeatedly	*Kits*	Lines 1771, 5209, 2037
Problem	There is no mechanism to detect whether state actually changed between saves. OnServerSave triggers on the vanilla server-save interval (default every 5 minutes). Even if not a single player redeemed a kit since the last save, the full serialise+backup cascade still runs. With 6 sibling plugins all doing the same in the same OnServerSave frame, the main thread stalls for the sum of all 6 cascades, stacked on top of Rust's own entity serialisation which is already the heaviest operation of any save tick.			
Fix	Add `private volatile bool _storageDirty;`. Set it true in UpdatePlayerData and buyclass. In OnServerSave and the periodic timer, guard: `if (!_storageDirty) return; _storageDirty = false;` before serialising. This ensures saves only occur when data has actually changed.			
Note	*Apply the same fix to all 6 sibling plugins simultaneously — the stall budget is the sum of all their save paths.*			

HIGH	[K-04]	ShiftBackups() runs on every save — excessive file ops in hot path	*Kits*	Line 6630 (KitsStorage.Save)
Problem	ShiftBackups() runs a full 5-level rename/copy cascade on every write — including the per-redemption saves identified in K-01. This means each kit claim does not just write one file; it also renames up to 5 backup files. On SSDs this is still measurably expensive at high frequency because the filesystem metadata operations are synchronous. The cascade also uses `File.Copy` followed by `File.Delete` rather than the atomic `File.Move`.			
Fix	Move ShiftBackups() out of the per-claim hot path entirely. With K-01 fixed (saves only on the periodic flush), backups only need to run at flush time, not per-action. Replace File.Copy+Delete with `File.Replace(tmp, path, backupPath)` for an atomic swap with one fewer syscall. Keep the backup cascade, just don't run it 80 times during a raid-night kit storm.			

MEDIUM	[K-05]	PlayerHasGroupPermission calls permission.UserHasPermission in overlay loop	*Kits*	Line 2222 (PlayerHasGroupPermission)
Problem	AppendPlayerOverlays() is called on every tab open and tab switch. Inside it, `PlayerHasGroupPermission()` calls `permission.UserHasPermission(player.UserIDString, kit.Permission)` directly (not via the sig cache) for every kit group. The developer already built a `ComputePermSignature` + `PlayerHasGroupPermBySig` system that avoids live permission calls — but he has two code paths and the overlay path uses the slow one.			
Fix	Replace all calls to `PlayerHasGroupPermission(player, ...)` inside AppendPlayerOverlays with `PlayerHasGroupPermBySig(sig, ...)` using the sig already computed at line 2700. The sig infrastructure is already there and correct — just use it consistently.			

MEDIUM	[K-06]	**String interpolation allocations in high-frequency UI overlay paths**	*Kits*	`Multiple — AppendPlayerOverlays, EmitRowOverlays, etc.`
Problem	Multiple locations inside AppendPlayerOverlays and its callees use C# string interpolation (`$"..."`) to build layer names and commands: `cardParent + $".Card.{kitsInGroup[0].ID}.Redeem.Dark"`, `parent + $".CD.{kit.ID}"`, etc. These allocate on every panel open. While ZString is used for the builder, the intermediate layer-name strings all go through the standard string allocator. On a 100-player server with everyone opening kits simultaneously, this is thousands of short-lived string allocations per second, sustaining GC pressure.			
Fix	Pre-compute and cache all static layer name strings at cache-build time. For dynamic names (per kit ID), use `ZString.Concat` or write directly into the `Utf8ValueStringBuilder` to avoid intermediate string objects. The developer is already using ZString for the JSON body — extend that discipline to layer names.			

2. PulseDailyGems — Critical Findings

Plugin: PulseDailyGems v2.0.2 · Author: Mevent.Team

CRITICAL	[DG-01]	**Synchronous disk save on every gem claim**	*PulseDailyGems*	Line 1460
Problem	CmdClaim() calls `_storage.Save(_storagePath)` synchronously after every claim. The same Save() cascade as Kits: full protobuf write + 5-deep backup shuffle + temp-file-copy pattern — all blocking the main thread. During the daily reset window when many players simultaneously claim gems, each claim stalls the main thread. Combined with K-01 in Kits and the same pattern in 4 other sibling plugins, a simultaneous reset storm can block the main thread for hundreds of milliseconds cumulatively.			
Fix	Identical fix to K-01: dirty flag + threaded write-behind. Claim sets `_storageDirty = true`, no disk I/O. FlushIfDirty() runs on OnServerSave, Unload, and timer.Every(120f). Serialise to byte[] on main thread; ThreadPool does all file operations under a lock. Durability window of 120s is acceptable for a daily-gems feature.			

HIGH	[DG-02]	**GetLastClaim() writes to _storage.Claims on every UI panel read**	*PulseDailyGems*	Lines 1548–1557
Problem	GetLastClaim() is called from AppendUiTo (every panel open) and from the claim-display path. When a player has no prior record, instead of returning 'claimable', the method inserts `_storage.Claims[userId] = (long)now` — setting the timestamp to NOW. This causes two bugs: (1) every new player who opens the panel is immediately put on a 24-hour cooldown having never received any gems; (2) the Claims dictionary grows every time any player opens the panel, making serialisation progressively heavier all wipe long.			
Fix	Return 0 for unknown players (fully claimable) without mutating state: `if (_storage.Claims.TryGetValue(userId, out var v) && v > 0) return v; return 0d;` Only write to Claims inside CmdClaim() after a successful claim. This also fixes the new-player cooldown bug.			
Note	*With this fix, Claims only grows when players actually claim gems, keeping the save file compact.*			

HIGH	[DG-03]	**No dirty flag — periodic OnServerSave writes unchanged data**	*PulseDailyGems*	Line 619
Problem	OnServerSave calls `_storage.Save()` unconditionally every server save tick. For PulseDailyGems this is especially wasteful: Claims only change when a player redeems, which happens at most once per player per 24 hours. The vast majority of OnServerSave calls write identical data.			
Fix	Same as K-03: guard with `if (!_storageDirty) return;`.			

MEDIUM	[DG-04]	**EncodeToPNG round-trip on image load — wasted CPU and GC at startup**	*PulseDailyGems*	`Line 1687`
Problem	LoadImage() downloads/reads each image via UnityWebRequestTexture, which decodes the PNG into a GPU Texture2D. It then calls `texture.EncodeToPNG()` which re-encodes the texture back to PNG bytes. This round-trip is wasteful: the original file is already valid PNG. EncodeToPNG allocates a large transient byte array for the recompressed data and triggers a GC event. For the 1.2 MB banner this is a 1.2+ MB allocation at load time, repeated for every image.			
Fix	Read the source file directly with `File.ReadAllBytes(path)` and pass the raw bytes to `FileStorage.server.Store()`. Skip UnityWebRequest and the Texture2D entirely. This removes the decode+encode cycle, cuts startup time, and avoids the GC spike. One caveat: if images are stored in non-PNG formats, they must be pre-converted to PNG offline.			

LOW	[DG-05]	**1.2 MB banner PNG is unoptimised**	*PulseDailyGems*	`Config: WelcomeBannerUrl`
Problem	The welcome banner is 1.2 MB (total images 1.8 MB). This does not cause server main-thread stalls but it does slow client first-open (the client must receive and decode the image on first panel open) and increases bandwidth consumption during wipe-day connect storms when many players connect simultaneously and all receive the image for the first time.			
Fix	Run all banner and background images through `pngquant --quality=70-90` and `oxipng -o 4`. A 1.2 MB banner can typically compress to 120–250 KB with no perceptible quality loss. This is an art pipeline task, not a code change.			

3. Architecture Assessment — What Works Well

Areas that are correctly implemented and should NOT be modified

Both plugins contain sophisticated, correct engineering in their UI rendering layers. The following are genuine strengths that distinguish this code from typical Oxide plugin work:

Binary protobuf-style persistence (both plugins)

Moving from JSON/DataFileSystem to a custom binary format with pooled BufferStream serialisation is the right architectural choice. The binary format is compact, fast to deserialise, and avoids Newtonsoft's reflection overhead on every read. The backup-rotation system is also correctly structured — the only problem is that it runs too frequently.

Pre-baked UTF-8 UI cache with byte-level marker splicing (both plugins)

The `CachedUi` / `UiTemplate` systems are excellent. UI JSON is built once into a byte array at load time. At send time, `AppendResolved` splices live values (online count, timers, use counts) directly using `ReadOnlySpan<byte>.CopyTo` — zero string allocation in the hot path. This is precisely the correct approach for high-frequency UI updates.

Raw RPC packet construction bypassing CuiHelper (both plugins)

`SendAddUiRaw` / `SendAddUiBytes` construct the AddUI RPC packet directly using `Network.Net.sv.StartWrite()` with a cached `StringPool.Get("AddUI")` ID. This avoids CuiHelper's string serialisation, JSON wrapping overhead, and the intermediate List<CuiElement> allocations. Correct.

Permission signature caching in Kits

The `ComputePermSignature / _sigPermIndex` system is well-designed: it maps kit permissions to bit positions and represents a player's permission set as a single ulong bitmask. UI templates can then be cached per-signature rather than per-player. This correctly eliminates O(players × kits) permission calls. The only issue is that one code path (K-05) bypasses it.

Facepunch.Pool usage throughout

Both plugins use `Pool.Get<List<T>>()` and `Pool.FreeUnmanaged(ref list)` correctly for temporary collections in the hot path. This is the right pattern and meaningfully reduces GC pressure versus allocating new List<T> instances per call.

Partial UI updates (not full rebuilds)

RefreshGemsCard, RefreshEventCardForAll, and HandleKitRedeemedUI all send only the changed sub-panel to the client — not the full UI hierarchy. On a 100-player server this is the difference between sending 1–2 KB vs 40–50 KB per update event. Correct.

4. Recommended Fix Order

Prioritised by impact on main-thread stall reduction

#	Finding	Plugin(s)	Expected Effect	Effort
1	K-01 + DG-01: Per-claim sync save → dirty flag + threaded flush	Both + all 6 siblings	**Removes claim-storm stalls entirely**	Medium
2	K-03 + DG-03: Unconditional OnServerSave flush → guard behind flag	Both + all 6 siblings	**Removes save-tick stacking**	Low
3	K-02: GetOrCreateKitData on read paths → GetOrLoadKitData	Kits	**Stops usersData growth; cuts serialise time**	Medium
4	DG-02: GetLastClaim write-on-read → read-only path	DailyGems	**Stops Claims growth; fixes new-player cooldown bug**	Low
5	K-04: ShiftBackups in hot path → move to flush only + File.Replace	Kits	Cuts syscalls per write from 15 to 4	Low
6	K-05: Use sig cache in overlay permission checks	Kits	Removes live permission calls from panel-open path	Low
7	K-06 + DG-05: Layer-name string alloc → ZString / pre-cached	Both	Reduces sustained GC pressure on high-pop	High
8	DG-04: EncodeToPNG round-trip → File.ReadAllBytes directly	DailyGems	One-time startup: removes large transient allocation	Low
9	DG-05: Compress banner images offline	DailyGems	Client connect-storm bandwidth (one-time per client)	None (asset prep)

Implementation note on threaded writes: Oxide runs on a Mono runtime where `System.IO` calls are thread-safe. `ThreadPool.QueueUserWorkItem` is safe for file I/O. The only main-thread-bound operation is reading from `Facepunch.Pool` (which requires main-thread affinity) — which is why the serialise-to-byte[] step must remain on the main thread, while only the file write step moves to the thread pool. Use a `lock(_ioLock)` guard to prevent concurrent writes from overlapping flushes.

5. Reference Code — Correct Persistence Pattern

Drop-in replacement for the per-claim Save() calls in both plugins

The following pattern applies identically to both Kits and PulseDailyGems. Adapt field names to match each plugin:

Fields to add:

```
private volatile bool _storageDirty; private readonly object _ioLock = new(); private Coroutine _flushTimer;
```

In OnServerInitialized():

```
// Bounded durability window — catches any missed dirty marks _flushTimer = timer.Every(120f, FlushStorageIfDirty);
```

Replace every inline `storage.Save()` call with:

```
// In GiveKitToPlayer / CmdClaim — after state mutation: _storageDirty = true; // mark dirty, do NOT touch disk here
```

Flush method (called by timer, OnServerSave):

```
private void FlushStorageIfDirty() { if (_storage == null || string.IsNullOrEmpty(_storagePath) || !_storageDirty) return; _storageDirty = false; // Serialise on main thread (Pool is main-thread bound, no I/O here) byte[] payload = SerialiseStorage(); var path = _storagePath; // All file operations on worker thread ThreadPool.QueueUserWorkItem(_ => { lock (_ioLock) { try { WriteWithBackups(path, payload); } catch (Exception e) { Interface.Oxide.LogError("[Kits] async save failed: " + e); } } }); }
```

Serialise to bytes on main thread:

```
private byte[] SerialiseStorage() { var stream = Pool.Get(); // main-thread only try { stream.Initialize(); _storage.WriteHeader(stream); _storage.WriteState(stream); var seg = stream.GetBuffer(); var bytes = new byte[seg.Count]; Buffer.BlockCopy(seg.Array, seg.Offset, bytes, 0, seg.Count); return bytes; } finally { Pool.Free(ref stream); } }
```

Atomic file write (worker thread):

```
private static void WriteWithBackups(string path, byte[] payload) { var dir = Path.GetDirectoryName(path); if (!string.IsNullOrEmpty(dir) && !Directory.Exists(dir)) Directory.CreateDirectory(dir); // Shift backups (5-deep rotate — now safe, runs off main thread) ShiftBackups(path); // Atomic write: write to .new, then replace var tmp = path + ".new"; File.WriteAllBytes(tmp, payload); // single write, no OpenWrite footgun File.Replace(tmp, path, path + ".bak"); // atomic swap on supported FS }
```

Unload must flush synchronously (plugin going away, no thread pool):

```
private void Unload() { if (_storageDirty && _storage != null && !string.IsNullOrEmpty(_storagePath)) { lock (_ioLock) { try { WriteWithBackups(_storagePath, SerialiseStorage()); } catch (Exception e) { PrintError("Kits unload save: " + e); } } } // ... rest of cleanup }
```

OnServerSave (same for both plugins):

```
private void OnServerSave() { FlushStorageIfDirty(); // no-ops if nothing changed }
```

Key Takeaway: The stalls you are experiencing are almost entirely caused by synchronous main-thread disk I/O in the persistence layer — not by UI rendering, image size, or network traffic. Fix K-01 and DG-01 (and apply the same pattern to all 6 sibling plugins) and the claim-storm stalls will disappear. Fix K-03 and DG-03 and the save-tick stacking will disappear. The remaining findings are genuine improvements but are secondary to these two changes.